

¿Qué es un Diagrama de Casos de Uso?

Un diagrama de casos de uso es un tipo de diagrama en UML que captura la funcionalidad y los requisitos de un sistema mediante la representación de actores y casos de uso. Los actores son entidades externas que interactúan con el sistema, mientras que los casos de uso representan las funcionalidades que el sistema debe ofrecer.

Elementos Clave de un Diagrama de Casos de Uso

Actores: Son entidades externas (personas, otros sistemas, etc.) que interactúan con el sistema. Se representan gráficamente con una figura de "muñeco de palo" y su nombre debajo. Un actor puede ser un usuario o un sistema que inicia una acción.

Casos de Uso: Representan funcionalidades específicas del sistema. Se dibujan como elipses con el nombre del caso de uso dentro. Cada caso de uso describe una acción que el sistema puede realizar en respuesta a la interacción de un actor.

Relaciones: Conectan actores con casos de uso y pueden incluir relaciones como <> (cuando un caso de uso utiliza otro) y <> (cuando un caso de uso puede extender el comportamiento de otro).

Importancia de los Casos de Uso

Identificación de Requisitos: Ayudan a identificar y documentar los requisitos funcionales del sistema desde la perspectiva del usuario, lo que es crucial durante la fase de análisis de requisitos.

Comunicación: Facilitan la comunicación entre desarrolladores y partes interesadas, proporcionando una visión clara de cómo se espera que funcione el sistema.

Guía para el Desarrollo: Sirven como punto de partida para el desarrollo del sistema, ayudando a los equipos a entender las funcionalidades que deben implementarse.

Cómo Dibujar un Diagrama de Casos de Uso

Identificar Funcionalidades: Analiza el sistema para identificar todas las funcionalidades que debe ofrecer.

Listar Actores: Enumera los actores que interactuarán con el sistema.

Definir Relaciones: Establece cómo interactúan los actores con los casos de uso y entre sí.

Utilizar Herramientas: Puedes usar herramientas como Lucidchart, Visual Paradigm o Draw.io para crear diagramas de manera eficiente.

Los diagramas de casos de uso son una herramienta esencial en el modelado de sistemas, proporcionando una representación clara y concisa de las interacciones entre los usuarios y el sistema.

Factores y sistemas de software por separado

Sistemas

Definición y Función

El **software de sistema**, también llamado software de base, es esencial para el funcionamiento de cualquier dispositivo digital. Su principal función es **administrar los recursos del hardware**, controlar dispositivos, ejecutar instrucciones y servir como base para que otros programas, como las aplicaciones, puedan operar. Actúa como un **punto de conexión entre el usuario, el hardware y las aplicaciones**, garantizando que el sistema sea funcional y estable.

Tipos de Software de Sistema

Sistemas operativos: Son el núcleo del software de sistema y coordinan todos los recursos del hardware, proporcionando una interfaz para que el usuario interactúe con el equipo. Ejemplos incluyen **Windows, macOS, Linux y Android**.

Controladores de dispositivos (drivers): Permiten que el sistema operativo se comunice con periféricos y componentes de hardware, como impresoras, tarjetas gráficas o cámaras.

Bootloaders o cargadores de programa: Programas que se ejecutan al encender un dispositivo y cargan el sistema operativo en la memoria, preparando el hardware para su uso.

Herramientas de diagnóstico y utilidades del sistema: Programas que monitorean, optimizan y protegen el sistema operativo y el hardware, ayudando a detectar fallos y mejorar la estabilidad.

BIOS y UEFI: Software que reside en la placa base y arranca el hardware al encender la computadora, entregando el control al sistema operativo.

Servidores y software de red: Gestionan la comunicación y los servicios dentro de redes locales o en la nube, asegurando conectividad y eficiencia.

Características

Características Clave

Fundamental para el sistema: Sin software de sistema, un computador o dispositivo sería inutilizable.

Interfaz de usuario: Aunque muchas funciones operan automáticamente, el software de sistema ofrece entornos de interacción, como escritorios o consolas de comandos.

Gestión de recursos: Administra CPU, memoria, almacenamiento y periféricos, optimizando el rendimiento y la estabilidad del sistema.

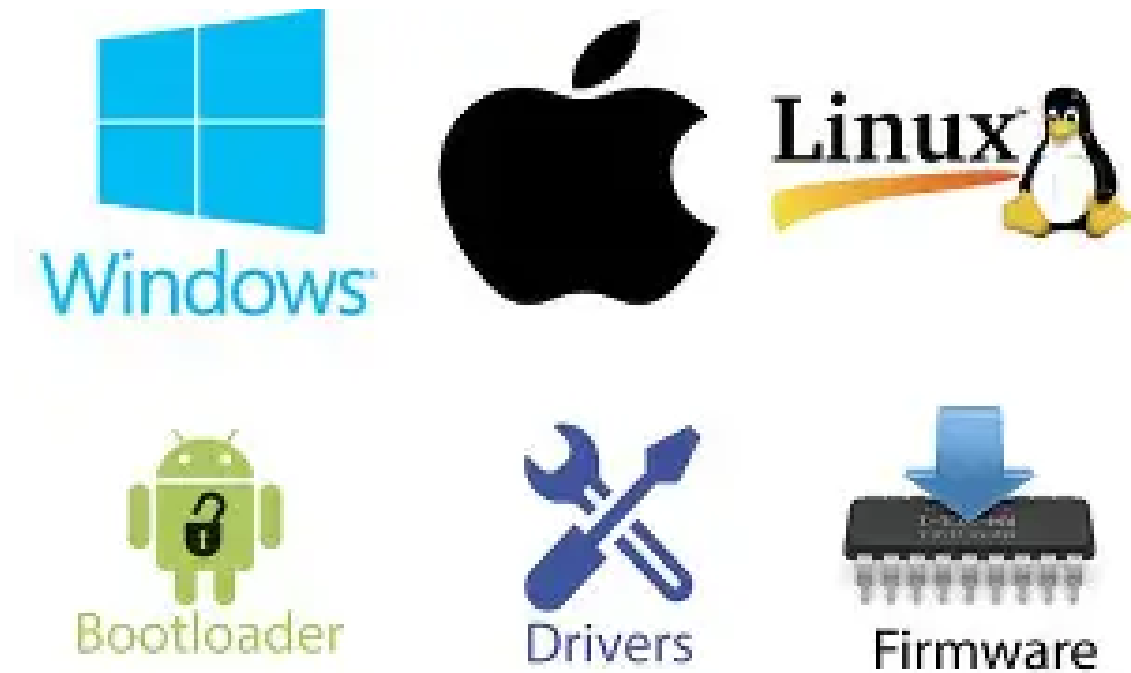
Seguridad y mantenimiento: Los sistemas modernos integran herramientas de seguridad y optimización, algunas con inteligencia artificial, para prevenir fallos y amenazas en tiempo real.

Importancia Actual

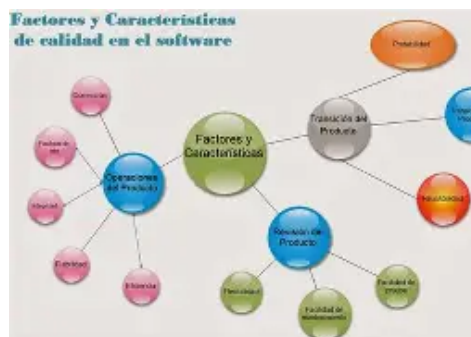
En 2025 y 2026, el software de sistema no solo mantiene la funcionalidad básica de los dispositivos, sino que también **optimiza recursos, integra IA, refuerza la seguridad y permite escalabilidad para empresas y startups**, reduciendo costos operativos y mejorando la eficiencia. Su dominio es clave para maximizar rendimiento y estabilidad en entornos digitales modernos.

Conclusión

En resumen, el **software de sistema es la base invisible que conecta hardware y aplicaciones**, asegurando que los dispositivos funcionen de manera eficiente, segura y adaptable a las necesidades del usuario y de las empresas.



Factores de software



Los factores de software son requisitos y atributos de calidad que condicionan el diseño de la arquitectura. Se dividen en **funcionales** (lo que el sistema hace) y **no funcionales** (cómo lo hace, bajo qué estándares de calidad).

En el **Sistema de Pedidos de Restaurante**, el problema principal es gestionar de forma eficiente y sin errores la comunicación entre el comedor (meseros), la cocina y la caja, asegurando que la información no se pierda y que el sistema sea fácil de usar en momentos de alta afluencia de clientes.

Mesero: Registra pedidos y consulta estados.

Cocinero: Visualiza pedidos pendientes y marca como finalizados.

Cajero: Procesa los pagos y cierra las órdenes.

Administrador: Gestiona el menú y genera reportes.

RF1: El sistema debe permitir al mesero crear una nueva orden asignada a una mesa.

RF2: El sistema debe enviar automáticamente el pedido a la pantalla de cocina.

RF3: El sistema debe permitir el registro de diferentes métodos de pago (efectivo, tarjeta).

Requisitos No Funcionales (RNF) alineados a ISO/IEC 25010

Usabilidad: La interfaz debe ser intuitiva para que un mesero aprenda a usarla en menos de 20 minutos.

Rendimiento: El tiempo de respuesta al guardar un pedido no debe superar los 2 segundos.

Seguridad: Solo los usuarios autorizados (login) pueden modificar estados de pedidos o precios.

Mantenibilidad: El código debe estar organizado de forma que añadir un nuevo tipo de platillo sea sencillo.

Se selecciona el patrón **MVC (Modelo-Vista-Controlador)**. **Justificación:** Permite separar la interfaz de usuario (Vista) de las reglas del negocio (Modelo) y de la gestión de datos (Controlador). Esto facilita que, si en el futuro el restaurante quiere cambiar sus tablets por una página web, solo se deba modificar la "Vista" sin tocar la lógica interna.

Capas: Presentación (UI), Lógica de Negocio y Persistencia (Base de Datos).

Componentes: Módulo de Pedidos, Módulo de Inventario, Módulo de Facturación.

Despliegue básico: Cliente (Tablet/PC) conectado mediante una red local a un Servidor.

Conclusión

la correcta identificación de los factores de software desde la **línea base arquitectónica** permite que el sistema de pedidos sea escalable y profesional. Aplicar la norma **ISO/IEC 25010** nos asegura que el software no solo sea útil, sino también confiable y fácil de mantener para el Instituto Nacional de Osicala.

Flujos alternos de software



Un **flujo alternativo** describe **qué pasa cuando el proceso principal no se sigue exactamente como se esperaba**. Es decir, muestra **variaciones o excepciones** al flujo normal de un sistema.

Flujo principal

El usuario ingresa su correo y contraseña.

El sistema valida los datos.

El sistema permite el acceso.

El usuario ingresa credenciales.

El sistema detecta que la contraseña es incorrecta.

El sistema muestra un mensaje de error.

El usuario puede intentar nuevamente.

El usuario ingresa un correo que no existe.

El sistema muestra mensaje: "usuario no registrado".

Se ofrece opción de registro.

Los **flujos alternos** son **caminos diferentes al flujo principal que describen qué ocurre cuando algo cambia o falla en un proceso del sistema**.

descripción de un caso de uso de un uml

La **descripción de un caso de uso en UML** es un documento que explica **cómo un usuario interactúa con un sistema para lograr un objetivo específico**. Se utiliza dentro del modelado de sistemas en Lenguaje Unificado de Modelado y en la Ingeniería de Software.

Un **caso de uso** describe **qué hace el sistema desde el punto de vista del usuario**, no cómo está programado.

1. Nombre del caso de uso

Identifica la acción principal del sistema.

Ejemplo:Registrar usuario

2. Actor

Es la persona o sistema externo que interactúa con el sistema.

Ejemplo:

Cliente / Usuario

3. Descripción

Explica brevemente qué hace el caso de uso.

Ejemplo:

Permite que un nuevo usuario cree una cuenta en el sistema ingresando sus datos personales.

4. Precondiciones

Condiciones que deben cumplirse antes de iniciar el caso de uso.

Ejemplo:

El usuario debe tener acceso a internet.

El sistema debe estar disponible.

5. Flujo principal (flujo normal)

Describe los pasos cuando todo funciona correctamente.

Ejemplo:

El usuario accede al formulario de registro.

El usuario ingresa sus datos (nombre, correo, contraseña).

El usuario presiona el botón “registrar”.

El sistema valida la información.

El sistema guarda los datos del usuario.

El sistema confirma el registro exitoso.

6. Flujos alternos

Son situaciones diferentes que pueden ocurrir.

Ejemplo:

A1 – Correo ya registrado

El sistema detecta que el correo ya existe.

El sistema muestra un mensaje: “correo ya registrado”.

A2 – Datos incompletos

El usuario deja campos vacíos.

El sistema solicita completar los campos obligatorios.

7. Postcondiciones

Describe el estado del sistema después de finalizar el caso de uso.

Ejemplo:

El usuario queda registrado en la base de datos.

El usuario puede iniciar sesión.

En resumen:

La descripción de un caso de uso en UML explica **actores, pasos del proceso, flujos alternos y condiciones**, permitiendo entender **cómo interactúan los usuarios con el sistema**.